

# Quinfosys™ QuDrugForge™ Frontend Application Report

---

## Table of Contents

1. [Executive Summary](#)
  2. [Product Purpose and Current Frontend Scope](#)
  3. [Current Application Architecture](#)
  4. [Global Layout Analysis](#)
  5. [Page-by-Page Frontend Analysis](#)
  6. [Research Module Analysis](#)
  7. [Visualization System Analysis](#)
  8. [AI Layer Analysis](#)
  9. [Infrastructure Pages Analysis](#)
  10. [Organization Pages Analysis](#)
  11. [Reusable Component Analysis](#)
  12. [Completed Work Summary](#)
  13. [Major Holes and Missing Functionality](#)
  14. [Testing Requirements](#)
  15. [Optimization Requirements](#)
  16. [Backend Integration Roadmap](#)
  17. [Priority Fix List](#)
  18. [Recommended Manual QA Checklist](#)
  19. [Final Evaluation](#)
  20. [Recommended Next Step](#)
- 

## 1. Executive Summary

**Quinfosys™ QuDrugForge™** is a high-fidelity frontend prototype designed for a Quantum AI Drug Discovery Platform. It implements a project-centric, computational drug discovery workflow tailored for oncology and molecular intelligence. The user interface establishes a sophisticated, clean, and light scientific aesthetic across a comprehensive array of software-as-a-service (SaaS) pages.

The current system relies entirely on mock data and static UI triggers. The application does not perform genuine authentication, file parsing, molecular docking, deep-learning inference, quantum reranking, ADMET prediction, or automated report generation. The strategic priority for the engineering team is **backend integration and route stabilization** rather than

expanding page count.

## Immediate Action Items

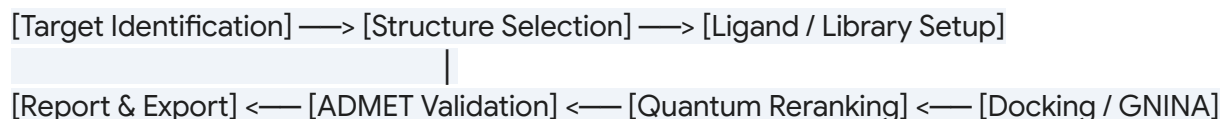
- Stabilize existing application routes and correct active sidebar states.
- Resolve the default dashboard fallback/offline rendering bugs.
- Verify every page component against multi-variant, realistic mock data profiles.
- Formally define backend API contracts (OpenAPI/Swagger schemas).
- Begin module-by-module data integration using abstract service layers.
- Deploy end-to-end (E2E) component testing for key user navigation flows.

---

## 2. Product Purpose and Current Frontend Scope

### 2.1 Intended Workflow Journey

The platform aims to orchestrate a complex pipeline for computational biologists, progressing through the following operations:



1. **Target & Input Specification:** Selection of disease indication (e.g., specific oncology targets), identification of genes/UniProt records, and ingestion of protein structures via FASTA, PDB, mmCIF, or AlphaFold repositories.
2. **Binding Site Definition:** Selection of localized residue coordinate frames or explicit bounding boxes alongside historical reference ligands.
3. **Compound Ingestion:** File processing for chemical libraries (SMILES, SDF, CSV) coupled with corresponding assay datasets ( $\text{\$IC}_{50}$ ,  $\text{\$K}_i$ ,  $\text{\$K}_d$ ,  $\text{\$EC}_{50}$ ).
4. **Multi-Tier Screening:** Execution of traditional molecular docking, CNN-based GNINA rescoring, and hybrid quantum machine learning (QML) electronic property reranking.
5. **ADMET & Stability Analysis:** High-throughput screening for toxicological parameters and trajectory physics evaluation through molecular dynamics (MD).
6. **Reporting:** Extraction of candidate dossiers, structural interaction fingerprints, and regulatory verification manifests.

### 2.2 Current Production Scope

The current iteration provides static visual layouts for nearly all milestones outlined above. It acts as an interactive user journey layout that maps user routing without exchanging data

payloads with high-performance computing clusters or persistent database architectures.

---

## 3. Current Application Architecture

### 3.1 Technical Stack Specifications

The frontend workspace leverages a modern TypeScript-Next.js stack built on the following primitives:

- **Core Framework:** React 18 / Next.js 14.2 (App Router)
- **Language Variant:** TypeScript 5.x
- **Styling Engine:** Tailwind CSS
- **State Coordination:** Zustand (Lightweight, non-boilerplate global slice management)
- **Animation Choreography:** Framer Motion
- **Data Aggregation Tables:** @tanstack/react-table (Headless virtualization engine)
- **Toast UI Notifications:** Sonner
- **Document Generation Engines:** react-markdown
- **Scientific Graphic/Visual Modules:**
  - react-plotly.js & Recharts (Vector and canvas mathematical data charting)
  - 3Dmol.js (WebGL particle acceleration engine for macromolecular models)

### 3.2 File System Architecture

```
src/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── register/
│   │   └── workspace-selector/
│   ├── (dashboard)/
│   │   ├── dashboard/
│   │   ├── research-projects/
│   │   ├── history/
│   │   └── results/
│   └── globals.css
├── components/
│   ├── dashboard/
│   ├── molecules/
│   └── shared/
```

```
| └─ ui/  
| └─ services/  
| └─ store/  
| └─ types/
```

## Structural Recommendation

As the platform prepares for backend onboarding, the team should transition towards a **feature-driven domain structure** to isolate complex scientific data handlers:

```
src/features/  
├─ projects/    # Program orchestration context  
├─ targets/     # UniProt/PDB integration modules  
├─ molecules/   # Structural properties, SMILES processing  
├─ docking/     # Autodock Vina / GNINA configurations  
└─ quantum/     # HOMO/LUMO descriptor parsers
```

## 4. Global Layout Analysis

The interface utilizes an AppShell container that uniformly enforces a standard workspace orientation across all internal dashboards.

### 4.1 Navigation Matrix (Sidebar)

The primary navigation architecture utilizes hierarchical grouping categorizing platform routes into distinct tiers:

| Category | Contained Route Modules                                                  |
|----------|--------------------------------------------------------------------------|
| Main     | Dashboard, Research Projects, Experiments (/history), Reports (/results) |
| Research | Targets, Molecules, Docking, GNINA, Quantum, Simulations, ADMET          |

|                |                                       |
|----------------|---------------------------------------|
| Visualization  | 3D Viewer, Chemical Space, Similarity |
| AI Layer       | Models, Pharma LLM (/copilot)         |
| Infrastructure | Compute, Storage, API, Integrations   |
| Organization   | Team, Billing, Audit Logs, Settings   |

### Critical Routing Defect

- **Symptom:** Certain active routes render content properly but trigger incorrect highlighting down the sidebar tree.
- **Root Cause:** Component relies on transient click listeners or un-normalized string lookups rather than active URL introspection.
- **Remediation:** Implement route-aware conditional matching based on the Next.js `usePathname()` hook.

TypeScript

```
const isActive = (route: string) => pathname.startsWith(route);
```

Nested profiles like `/research-projects/[id]` must evaluate via partial prefix criteria, and query-parameter-driven settings tabs must read context keys from the search parameters object.

## 4.2 Application Topbar

The global layout crown controls persistent macro actions:

- **Global Context Query:** String parsing search bar (currently unmapped to a command engine).
- **Global Project Badge:** Visual tracking chip identifying the active discovery scope (e.g., EGFR NSCLC Active).
- **System Indicators:** Status triggers for notification events, light/dark theme toggles, and

user contextual profiles.

## 5. Page-by-Page Frontend Analysis

### 5.1 Landing & Product Presentation

- **Functional Goal:** Public facing product value exposition and sign-up conversion gateway.
- **Present Elements:** Dynamic hero sections, interactive platform workflows, key metric showcases, molecular stack breakdowns, and call-to-action (CTA) triggers.
- **Engineering Deficiencies:** Lacks an integrated tier-pricing grid, functional interactive video walk-through placeholders, and SEO/Meta indexing headers. Landing modules are bundled alongside internal routes, risking structural middleware leakage.

### 5.2 Authentication (Login, Registration, Workspace Selection)

- **Functional Goal:** Secure workspace entry point and onboarding engine.
- **Present Elements:** Split-screen layouts containing WebGL particle backgrounds on the left flank; standard login credentials, organization parameters, role settings checkboxes, and mock workspace selectors on the right.
- **Engineering Deficiencies:** Session management layers do not exist. Forms lack client-side syntactic regex validation, error boundary mappings, and secure password-strength meters. Federated single-sign-on (SSO) nodes remain purely cosmetic layout tokens.

### 5.3 Central Discovery Dashboard

- **Functional Goal:** Aggregating critical alerts, current pipeline tracking, active compute profiles, and direct workflow execution entries.
- **Present Elements:** Target progress step indicators, recent telemetry cards, timeline charts, and an inline Pharma LLM side drawer.

|                                |                   |                          |
|--------------------------------|-------------------|--------------------------|
| +-----+                        |                   |                          |
| [Active Project: EGFR NSCLC]   | (User Profile)    |                          |
| +-----+                        |                   |                          |
| [Metric: 2.4k]                 | [Metric: 92%]     | [Metric: 14 Active Jobs] |
| +-----+                        |                   |                          |
|                                |                   |                          |
| [Pipeline Visualization Chart] | [Pharma LLM Chat] |                          |
|                                |                   |                          |

|                               |  |  |
|-------------------------------|--|--|
| [Recent Telemetry Event Logs] |  |  |
|                               |  |  |
| +-----+                       |  |  |

- **Critical Defect:** The production workspace occasionally renders an aggregate systemic failure component (Dashboard System Offline) as its base layout variant. This occurs because the initial boundary wrapper evaluates an unset operational boolean to false or catches a network error on an unmocked fetch sequence. The system must default gracefully to standard mock schemas.

## 5.4 Research Project Space & Tab Workspace

- **Functional Goal:** Unified operational context manager for discovery workflows (e.g., PARP1, PIK3CA, KRAS programs).
- **Present Elements:** Index layouts displaying program health metrics, owner allocation metrics, and chronological action streams.
- **Engineering Deficiencies:** Creation dialog interfaces do not write mutations back to local state stores. Transition points from decoupled pages into unified project tabs require strict dynamic routing consolidation:

/research-projects/[projectSlug]/[moduleTab]

## 6. Research Module Analysis

### 6.1 Biological Targets

- **Functional Goal:** Management of biological targets, pathway visualization, and structural structure availability tracking.
- **Present Elements:** Tabular views tracking UniProt indicators, structural identification statuses, and binding pocket coordinate tables.
- **Engineering Deficiencies:** No dynamic queries linked to ChEMBL, PubChem, or NCBI REST interfaces. Structure uploads fail to instantiate file validations.

### 6.2 Small Molecule Library Screening

- **Functional Goal:** Ingestion, tabular structure tracking, and quantitative chemical evaluation.
- **Present Elements:** Data frameworks logging Molecular Weight (MW), Partition

Coefficients (\$\log P\$), Quantitative Estimate of Drug-likeness (QED), and Topological Polar Surface Area (TPSA).

- **Engineering Deficiencies:** No client-side chemoinformatics framework integration (e.g., RDKit compilation via WebAssembly). The system cannot process SMILES notations or compute actual fingerprint matrix configurations.

### 6.3 Classical Docking & Deep-Learning GNINA Rescoring

- **Functional Goal:** Execution and analysis of classical binding affinities alongside neural-network affinity inferences.
- **Present Elements:** Multi-variable form controllers for pocket box coordinate definitions, docking run submission parameters, and tables displaying mock affinity scores.
- **Engineering Deficiencies:** Interface does not route computing payloads to back-end execution managers. GNINA rescoring parameters exist as a query property variation of the primary docking route (/docking?engine=gnina), which causes side-navigation desynchronization.

### 6.4 Quantum Machine Learning (QML) Evaluation

- **Functional Goal:** High-fidelity molecular property calculations and quantum-tier candidate optimization.
- **Present Elements:** Graphical representations of Highest Occupied Molecular Orbital (HOMO), Lowest Unoccupied Molecular Orbital (LUMO), orbital band gaps, and overall QML structural fitness scores.
- **Engineering Deficiencies:** Lacks computational backing from semi-empirical solvers (e.g., xTB, GFN2) or structural geometry optimizing sub-routines.

### 6.5 Molecular Dynamics Simulation & ADMET Analysis

- **Functional Goal:** Evaluating structural binding longevity alongside comprehensive pharmacokinetic risk verification.
- **Present Elements:** Interactive canvas areas for trajectory playback, multi-axis charting profiles showing Root Mean Square Deviation (RMSD) and Root Mean Square Fluctuation (RMSF), and radar chart components tracking absorption, distribution, metabolism, excretion, and toxicity profiles.
- **Engineering Deficiencies:** Trajectory elements lack structural file bindings (e.g., DCD, XTC formats). ADMET routes are coupled to query panels (/validation?panel=admet), preventing clean direct linking.

---

## 7. Visualization System Analysis

### 7.1 Structural Discovery Workbench (3D Viewer)



- **Functional Goal:** High-performance, WebGL-accelerated 3D inspection of complex macro-proteins and small-molecule binding configurations.
- **Present Elements:** Canvas display containers, basic representation toggles (ribbon, cartoon, spacefill), atom/residue label queries, and side context panels.
- **Engineering Deficiencies:** The structural viewing area displays loading placeholders without parsing raw coordinate files (\$PDB\$, \$mmCIF\$, \$SDF\$). Interaction fingerprint lines between ligand structures and target residues are hard-coded vectors.

## 7.2 Multi-Dimensional Chemical Space Mapping

- **Functional Goal:** Dimensionality reduction layout illustrating compound diversity, cluster boundaries, and scaffold distribution.
- **Present Elements:** High-density 2D scatter plots mapping spatial layouts intended to reflect output projections from algorithms like UMAP or \$t-SNE\$.
- **Engineering Deficiencies:** Coordinates are generated by basic uniform random functions. Hover modules fail to draw instantaneous chemical structures or resolve cluster categorization queries.

## 7.3 Structural Similarity Frameworks

- **Functional Goal:** Calculating and visualizing cross-library candidate similarities against control active ligands.
- **Present Elements:** Symmetrical comparison matrices, Tanimoto similarity trackers, and structural clustering trees.
- **Engineering Deficiencies:** Calculations do not dynamically update upon chemical state alteration due to a lack of live fingerprinting infrastructure.

---

# 8. AI Layer Analysis

## 8.1 Model Registry & Monitoring Dashboard

- **Functional Goal:** Version auditing, training lineage configuration, and prediction validation oversight.
- **Present Elements:** Target loss charts, accuracy metric grids, feature weights tracking panels, and active production endpoint status fields.
- **Engineering Deficiencies:** Registry interfaces lack data synchronization with model tracking platforms (such as MLflow) or active container endpoints.

## 8.2 Pharma LLM / Contextual Copilot

- **Functional Goal:** Conversational interface assisting workflows, querying scientific literature, and generating synthesis blueprints.
- **Present Elements:** Chat thread containers, structural prompt chips, and rich text output

cards layout panels.

- **Engineering Deficiencies:** Dialogue interaction points do not communicate with an active LLM inference engine. Contextual state retrieval mechanisms cannot parse workspace parameters or generate citations.

---

## 9. Infrastructure Pages Analysis

The system currently groups technical infrastructure operations within a unified configuration path using explicit search parameter selectors.

```
    /—> ?section=compute —> GPU Clusters / Queues
    |—> ?section=storage —> Datasets / S3 Buckets
/settings?section=... |—> ?section=api —> Keys / Webhooks
    |—> ?section=integrat —> AlphaFold / ChEMBL Nodes
```

### 9.1 Infrastructure Profile Definitions

| +-----+      |                                |                       |
|--------------|--------------------------------|-----------------------|
| Sub-Module   | Visual Elements Implemented    | Integration Gap       |
| +-----+      |                                |                       |
| Compute      | GPU usage meters, queue state  | No live hardware logs |
| Storage      | File capacity graphs, tables   | Missing AWS S3/Blob   |
| API Portal   | Token rows, mock code blocks   | Cryptographic storage |
| Integrations | Active/Inactive vendor toggles | Credentials handshake |
| +-----+      |                                |                       |

- **Compute Workspace:** Displays live cluster utilisation graphics and resource queue arrays. Currently operates without connections to resource balancers (e.g., Slurm) or cloud orchestration planes.
- **Storage Hub:** Tracks global account capacity usage, dataset integrity indexes, and upload event arrays. Lacks integration with target object storage buckets (S3/Azure Blob).
- **Developer API Portal:** Manages functional interface token parameters, endpoint usage trackers, and documentation panels. Tokens are generated locally without security checks or authorization tracking.
- **Integrations Node:** Lists toggle frameworks for third-party platforms (AlphaFold,

HuggingFace, Milvus). Connections are simulated and do not trigger remote handshakes.

---

## 10. Organization Pages Analysis

- **Team Workspace Management:** Provides tabular interfaces for inviting members, altering security roles (e.g., Principal Investigator, Research Assistant), and configuring domain access rights. Lacks integration with access management (RBAC) engines.
- **Billing Ledger:** Displays consumption invoice logs, credit breakdown graphics, and plan option grids. No payment gateways (e.g., Stripe) are linked.
- **System Audit Tracking:** Renders security event streams, data mutation indicators, and cryptographic hash verification logs. Logs are stored in transient state arrays and clear upon page reload.
- **System Settings:** Controls user interface configurations, metadata parameters, notification settings, and system-wide modifications. Configuration adjustments are not persisted to local storage or user database models.

---

## 11. Reusable Component Analysis

### 11.1 Reusable UI Catalog

| Component Token | Production Readiness / Current Status         | Action Items & Required Enhancements                          |
|-----------------|-----------------------------------------------|---------------------------------------------------------------|
| AppShell        | Structure validated across various viewports. | Resolve nested layout scroll loops; normalize overlay panels. |
| Sidebar         | Layout groups are clearly delineated.         | Transition logic to route-aware matching hooks.               |
| Topbar          | Workspace profile features are complete.      | Wire search queries directly into the CommandPalette.         |

|                |                                                 |                                                               |
|----------------|-------------------------------------------------|---------------------------------------------------------------|
| MetricCard     | Uniform layout structures are established.      | Implement standard data skeleton utilities for hydration.     |
| DataTable      | Core headless framework is properly configured. | Standardize client pagination handlers and column grouping.   |
| CommandPalette | Modal structures capture keyboard shortcuts.    | Integrate structural fuzzy searching across all routes.       |
| StateAlerts    | Error/Empty visual layouts are available.       | Address conditional logic rendering dashboard offline errors. |

## 12. Completed Work Summary

The following milestones have been successfully achieved from a design and interface perspective:

### 12.1 Visual and Structural Milestones

- Complete page coverage mapping across all 26 core operational interfaces.
- Consistent application of a professional scientific design system with clear typography and balanced visual contrast.
- Highly organized sidebar layout that mirrors real-world computational biology pipelines.
- Clean responsive layout configurations utilizing standard Tailwind structural grid definitions.
- Advanced implementation of data visualization tables using @tanstack/react-table.

### 12.2 Conceptual Alignment

- Clear project-centric orientation rather than fragmented individual utilities.
- Logical progression through sequential research modules (Docking  $\rightarrow$  GNINA  $\rightarrow$  Quantum  $\rightarrow$  ADMET  $\rightarrow$  Reports).
- Integration of advanced AI assistance layers across different sections of the user experience.

---

## 13. Major Holes and Missing Functionality

### 13.1 Core Integration Architectural Voids

The platform's primary gap is the **complete absence of a functional service abstraction layer**. The system cannot parse structural files or maintain data persistence across active operational domains.

### 13.2 Route Structural Discrepancies

The application contains routing differences between its sidebar naming standards and actual URL parameters:

- Sidebar item Experiments targets structural route `/history`.
- Sidebar item Reports targets structural route `/results`.
- Scientific routes like GNINA and ADMET exist as query transformations rather than isolated routing segments (`/docking?engine=gnina`, `/validation?panel=admet`).

#### Structural Recommendation

Normalize application paths before connecting backend endpoints to ensure predictable route state mapping:

```
/dashboard  
/research-projects/[id]  
/experiments/[id]  
/reports/[id]  
/gnina  
/admet
```

---

## 14. Testing Requirements

[Navigation Matrix] —> E2E Path Validation —> Verify Sidebar Highlight Status  
[Authentication] —> Boundary Assertions —> Validate Regex & Fail Conditions

[Scientific Inputs] —> Payload Verification —> Check PDB / SMILES File Parsing

## 14.1 Manual Navigation Matrix

Every route module must undergo automated E2E testing using tools like Playwright to confirm structural health:

- Confirm zero browser console exceptions or unresolved promise warnings during route transitions.
- Verify sidebars accurately match active route configurations.
- Ensure that resizing viewports down to tablet scale triggers structural collapse menus cleanly without throwing layout overflow errors.

## 14.2 Authentication Boundary Assertions

- Assert that submissions with empty input values or invalid email formats trigger localized error states instead of sending network payloads.
- Verify that direct unauthenticated page routing redirection loops unauthorized agents back to the login gateway.

## 14.3 Technical Dataset Payload Verification

The pipeline must undergo verification testing using authentic bioinformatic data files:

- **Macromolecules:** Validate parsing behavior for standard structural files, including crystalline coordinate sets for target receptors (e.g., standard wild-type EGFR structures).
- **Chemical Compounds:** Process standard small-molecule SMILES arrays and verify compound list ingestion parsing.
- **Analytical Telemetry:** Verify chart performance when rendering high-frequency data structures, such as 10,000+ data point trajectory data streams within the MD simulation modules.

---

# 15. Optimization Requirements

## 15.1 UI/UX Refinements

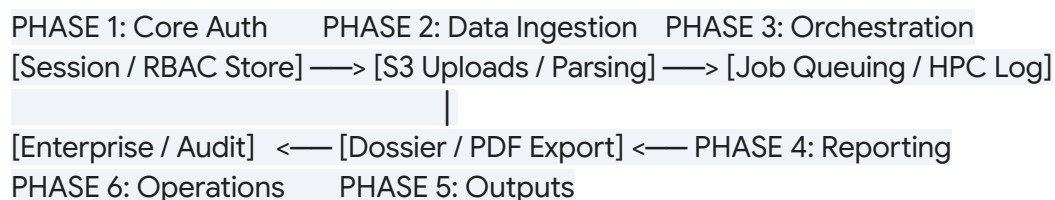
- Standardize padding values across all page headers to eliminate layout shifts during route transitions.
- Address minor card dimension issues on high-resolution displays to maintain uniform grid alignments.
- Ensure consistent row density profiles across all tables.
- Verify and improve color contrast ratios in both light and dark modes to meet WCAG AA standards.

## 15.2 Code and Performance Tuning

- Extract large mock data objects out of component render cycles and store them in separate mock asset files.
- Wrap complex client-side calculations (such as search indexing filters) in standard React optimization hooks (useMemo).
- Implement dynamic code splitting and lazy-loading for heavier visualization libraries (3Dmol.js, Plotly).
- Prevent components inside inactive workspace tabs from rendering until their respective tab becomes active.

---

## 16. Backend Integration Roadmap



### Phase 1: Authentication and Session Layer

- Implement production authentication systems utilizing standard JWT transport strategies or OAuth/OIDC handlers.
- Enforce tenant-isolated routing controls within Next.js middleware components.

### Phase 2: Ingestion Pipelines and Persistent Projects

- Deploy secure S3/Blob storage upload flows using presigned URL handshakes.
- Build backend file parsing engines to process uploaded structural assets and populate internal data stores.

### Phase 3: Analytical Data Ingestion

- Integrate centralized databases (such as PostgreSQL) with specialized chemoinformatics extensions (e.g., RDKit extensions) to handle structural data.
- Expose endpoints for molecule screening, structural property lookup, and similarity querying.

### Phase 4: Compute Orchestration Linkage

- Connect asynchronous job workers (such as Celery or Redis Task Queues) to broker docking workloads with underlying HPC resources.
- Expose streaming connections (such as WebSockets or Server-Sent Events) to feed real-time compute telemetry back to the dashboard interface.

## Phase 5: Automated Dossier Assembly

- Deploy server-side PDF rendering architectures to automatically compile research evidence packages.
- Implement export features for structured scientific file formats (SDF, CSV).

## Phase 6: Enterprise Management Integrations

- Connect billing engines with cloud usage meters to dynamically adjust account balances based on active cluster compute consumption.
- Route operational log entries directly to secure, append-only security auditing systems.

---

# 17. Priority Fix List

## Critical Priority (Immediate Action Required)

1. **Resolve Dashboard Rendering Fault:** Modify conditional statements inside `dashboard/page.tsx` to prevent the platform error state component from rendering by default.
2. **Fix Active Navigation Indicators:** Refactor the sidebar rendering loop to derive focus indicators directly from Next.js routing paths instead of tracking clicked states.
3. **Establish Interface Access Bounds:** Isolate marketing landing paths from internal workspace applications to protect authenticated sessions.

## High Priority

1. **Consolidate Project Routing Structures:** Migrate isolated tool views into an integrated, parameter-driven tab environment under unified program workspaces.
2. **Implement Safe File Upload Interfaces:** Upgrade file upload zones with strict validation rules governing size limits and allowable scientific extensions.
3. **Introduce Skeletons for Loading States:** Replace default full-page loaders with precise skeleton layout components during data fetching states.

## Medium Priority

1. **Enable Command Palette Functionality:** Wire the global shortcut listener to an active search index engine to enable rapid cross-route navigation.
2. **Build Out Item Detail Views:** Create dedicated slide-out panels and individual details sub-pages for specific compound and experiment profile rows.



---

## 18. Recommended Manual QA Checklist

### Global App Shell

- ☐ Verify the sidebar smoothly collapses and expands without causing chart layout glitches.
- ☐ Check that your currently active sidebar category perfectly matches your active browser path.
- ☐ Confirm that hitting Ctrl + K or Cmd + K reliably calls up the global search overlay.
- ☐ Test the theme switcher to ensure seamless swapping between light and dark presentation layers.

### Authentication Pathways

- ☐ Confirm submitting empty email or password fields triggers localized validation warnings.
- ☐ Verify that forcing an unauthorized direct route jump routes the agent back to the sign-in screen.
- ☐ Ensure the main sidebar navigation elements are hidden while browsing authentication forms.

### Scientific Research Profiles

- ☐ Verify that structural metrics layout cards display readable values across both narrow and wide displays.
- ☐ Test data tables to ensure column sorting triggers smoothly without throwing runtime errors.
- ☐ Confirm that long strings (like extensive protein sequence identifiers) warp cleanly without breaking parent container borders.

### Alerting System Testing

- ☐ Trigger mock error statuses to verify components handle failure responses gracefully.
- ☐ Verify that toast notification messages render correctly with appropriate color codes matching their error or success status.

---

## 19. Final Evaluation

Quinfosys™ QuDrugForge™ features a well-designed, enterprise-ready user interface layout tailored for modern bioinformatic SaaS solutions. Its comprehensive workflows and clean design aesthetic look highly professional.

| METRIC                   | SCORE (1-10)    |
|--------------------------|-----------------|
| Interface Aesthetics     | 9.5             |
| Workflow Coverage        | 9.0             |
| Component Reusability    | 8.0             |
| Routing Design           | 5.0             |
| Backend Data Integration | 0.0 (Prototype) |

The underlying system remains entirely uncoupled from live data engines and real-world calculation layers. It currently operates as an interactive wireframe. Transitioning the platform into a secure, production-grade application will require systematic route stabilization, robust data model definitions, and the implementation of a feature-driven backend integration roadmap.

## 20. Recommended Next Step

The engineering team should **halt the creation of additional front-end pages** and immediately pivot to **Phase A of the Backend Integration Roadmap**.

### Immediate Milestone: Standardizing the API Layer

Architect an API specification document outlining necessary JSON structures, data types, and status code behaviors.

TypeScript

```
// Proposed TypeScript Model for Platform Structural Data Coordination
```

```
export interface MoleculeCandidate {
```

```
  id: string;
```

```
  projectId: string;
```

```
  smiles: string;
```

```
  iupacName: string;
```

```
  molecularWeight: number;
```

```
  logP: number;
```

```
qed: number;
tpsa: number;
dockingScore?: number;
gninaConfidence?: number;
qmlFitness?: number;
admetRisks: Array<'hERG' | 'CYP' | 'BBB' | 'Hepatotoxicity'>;
status: 'draft' | 'queued' | 'processing' | 'validated' | 'failed';
createdAt: string;
}
```

Prior to building out the actual calculation infrastructure, engineers should configure a flexible client framework that allows the UI to easily toggle between local mock data files and live development server endpoints:

### TypeScript

```
// Environment-Driven API Target Switch
const API_BASE_URL = process.env.NEXT_PUBLIC_API_MODE === 'production'
  ? 'https://api.qudrugforge.quinfosys.com/v1'
  : '/api/mock-provisional';
```